1. Introduction

This project presents a software system for analyzing the Gene Ontology (GO) and human gene annotations. The system processes biological datasets provided in OBO and GAF formats and builds an internal representation that allows users to explore ontology structures, analyze gene annotations, and compute similarity between genes. The application integrates data parsing, graph-based modelling, numerical computation, and a web-based interface, providing a complete environment for interacting with biological data.

---

2. System Overview

The system operates on two main datasets: the Gene Ontology, which defines biological concepts and their hierarchical relationships, and the Gene Annotation File, which links genes to GO terms. After parsing these files, the system constructs a graph-based representation in which GO terms are modelled as nodes connected through parent–child relationships, forming a directed acyclic graph.

Gene annotations are associated with these terms, allowing each gene to be linked to multiple functional concepts. This structure enables the system to support various operations, including retrieving the annotations of a gene, exploring the ancestors and descendants of a GO term, and computing similarity between genes based on shared ontology information.

The user interacts with the system through a web interface implemented using Flask. Through this interface, users can search for genes and GO terms, explore ontology relationships, and visualize results, including graphical representations of ontology substructures.

---

3. Project Structure

The project is organized into multiple modules, each responsible for a specific aspect of the system. The core functionality is implemented in Python files that handle data parsing, ontology modelling, annotation management, and analytical computations. The web interface is implemented using Flask, with HTML templates stored in a dedicated `templates` folder.

The repository also includes the UML class diagram and CRC cards used to document the system design, as well as example data files for testing. This modular organization improves readability, maintainability, and extensibility.

---

4. System Architecture

The system is designed as a modular architecture in which different components handle specific responsibilities while interacting in a coordinated manner. The parsing component is responsible for reading and interpreting the input datasets through a common abstract interface and specialized implementations for ontology and annotation files. The parsed data is stored using Pandas DataFrame, ensuring structured and efficient access.

The ontology component models the Gene Ontology as a graph structure. Each GO term is represented as an object containing its attributes and relationships, while a central ontology class manages the collection of terms and supports navigation operations such as retrieving ancestors, descendants, and computing term depth. This design enables efficient traversal of the ontology graph.

The annotation component manages the relationships between genes and GO terms. Individual annotations are represented as objects, while a dedicated container organizes them by gene, allowing efficient retrieval and preventing duplication. This structure supports fast queries when analyzing gene-related information.

The analytical component provides methods for comparing genes based on their associated ontology terms. Similarity between genes is computed using interchangeable strategies, allowing different similarity measures to be applied without modifying the core logic.

The behavior of the system is implemented through key methods operating on these components. For example, the ontology provides methods such as `get_ancestors()` and `get_descendants()` for navigating the graph structure. The analytical component includes methods such as `gene_similarity()` and `similarity_matrix()`, which integrate ontology and annotation data to compute meaningful similarity scores between genes.

Finally, the system includes a web interface implemented using Flask, which connects all components. It handles user requests, triggers backend computations, and renders results through HTML templates. Graph visualization is supported using NetworkX and Matplotlib, allowing users to explore ontology structures visually.

---

5. Design Decisions

The design of the system is guided by the principles of modularity, clarity, and extensibility. The system is decomposed into independent components, each responsible for a specific aspect of the functionality, such as data parsing, ontology representation, annotation management, and analysis. This separation of concerns reduces coupling and improves maintainability.

Object-oriented principles are applied throughout the design. Encapsulation is achieved by ensuring that each class manages its own data and exposes only the necessary operations. For

example, the ontology structure is internally maintained within a dedicated class, which provides controlled access through well-defined methods.

Abstraction is implemented through abstract base classes that define common interfaces for parsers and similarity measures, allowing different implementations without exposing internal details. Inheritance is used to extend and specialize behavior, while polymorphism is demonstrated in the analytical component, where different similarity measures can be used interchangeably.

The system integrates multiple data structures to handle different types of operations efficiently. Pandas DataFrame are used for parsing and preprocessing data, sets and dictionaries support efficient lookup operations, NumPy matrices are used for numerical computations, and graph structures are used for ontology representation and visualization.

A key design decision was to focus on `is_a` relationships within the ontology, simplifying the implementation while preserving the essential hierarchical structure. The architecture remains extensible, allowing future integration of additional relationship types and analytical methods.

---

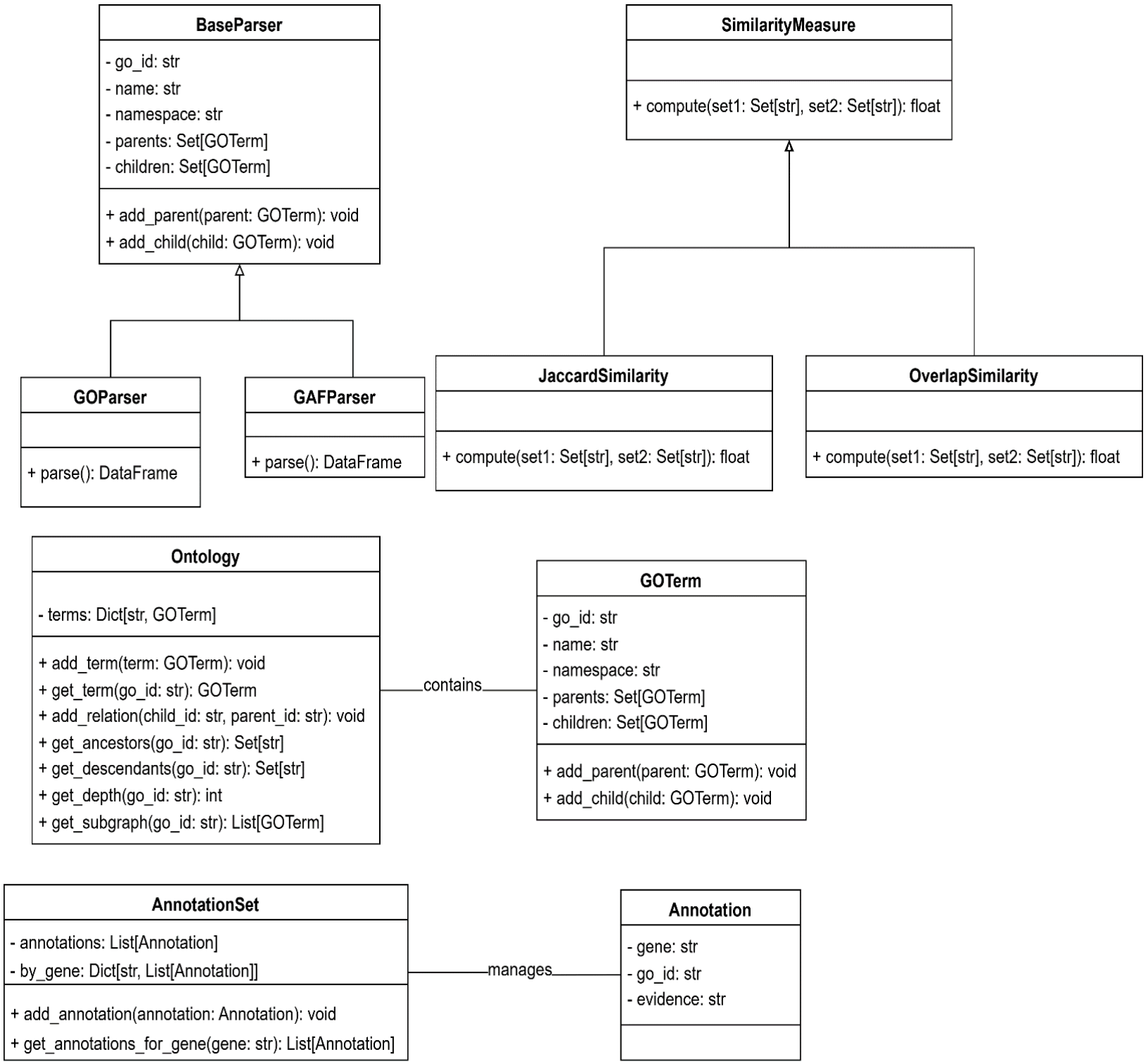
6. Conclusion

The developed system provides a modular and extensible solution for analysing gene ontology data. By combining structured data processing, object-oriented modelling, and an interactive web interface, it enables users to explore biological relationships and perform analytical tasks effectively. The clear separation of components ensures maintainability and supports future extensions.

7. UML Diagram and CRC Cards

The following diagrams illustrate the structural and behavioural design of the system. The UML class diagram provides an overview of the main components, their relationships, and the use of object-oriented principles such as inheritance and association.

The CRC (Class–Responsibility–Collaborator) cards complement the UML diagram by describing the responsibilities of each class and their interactions with other components. Together, these representations provide a clear understanding of the system architecture and design rationale.


BaseParser
- go_id: str
- name: str
- namespace: str
- parents: Set[GOTerm]
- children: Set[GOTerm]
+ add_parent(parent: GOTerm): void
+ add_child(child: GOTerm): void
SimilarityMeasure
+ compute(set1: Set[str], set2: Set[str]): float
GOParser
+ parse(): DataFrame
GAFParser
+ parse(): DataFrame
JaccardSimilarity
+ compute(set1: Set[str], set2: Set[str]): float
OverlapSimilarity
+ compute(set1: Set[str], set2: Set[str]): float
Ontology
- terms: Dict[str, GOTerm]
+ add_term(term: GOTerm): void
+ get_term(go_id: str): GOTerm
+ add_relation(child_id: str, parent_id: str): void
+ get_ancestors(go_id: str): Set[str]
+ get_descendants(go_id: str): Set[str]
+ get_depth(go_id: str): int
+ get_subgraph(go_id: str): List[GOTerm]
contains
GOTerm
- go_id: str
- name: str
- namespace: str
- parents: Set[GOTerm]
- children: Set[GOTerm]
+ add_parent(parent: GOTerm): void
+ add_child(child: GOTerm): void
AnnotationSet
- annotations: List[Annotation]
- by_gene: Dict[str, List[Annotation]]
+ add_annotation(annotation: Annotation): void
+ get_annotations_for_gene(gene: str): List[Annotation]
manages
Annotation
- gene: str
- go_id: str
- evidence: str

Class name: BaseParser	**Superclass: —**	**Subclasses:** GOParser, GAFParser
Responsibilities	**Collaborations**	
Define parsing interface	GOParser	
Store file path		
Store parsed data	GAFParser	
Provide data access		

Class name: GOParser	**Superclass:** BaseParser	**Subclasses: —**
Responsibilities	**Collaborations**	
Parse GO OBO file	BaseParser	
Extract GO terms		
Extract relationships	Ontology	
Build DataFrame		

Class name: GAFParser	**Superclass:** BaseParser	**Subclasses: —**
Responsibilities:	**Collaborations:**	
Parse GAF file	BaseParser	
Extract gene annotations		
Ignore comment lines	AnnotationSet	
Build DataFrame		

Class name: GOTerm	**Superclass: —**	**Subclasses: —**
Responsibilities:	**Collaborations:**	
Represent GO term	Ontology	
Store ID, name, namespace		
Maintain parent terms	GOTerm	
Maintain child terms		

Class name: Ontology	**Superclass: —**	**Subclasses: —**
Responsibilities:	**Collaborations:**	
Store GO terms	GOTerm	
Build term relationships		
Retrieve term by ID		
Compute ancestors		
Compute descendants		
Compute depth		

Class name: Annotation	**Superclass: —**	**Subclasses: —**
Responsibilities:	**Collaborations:**	
Represent gene annotation	AnnotationSet	
Store gene identifier		
Store GO ID		
Store evidence code		

Class name: AnnotationSet	**Superclass: —**	**Subclasses: —**
Responsibilities:	**Collaborations:**	
Store annotations	Ontology	
Group annotations by gene		
Prevent duplicates	Annotation	
Retrieve annotations by gene		

Class name: SimilarityMeasure	**Superclass: —**	**Subclasses:** JaccardSimilarity, OverlapSimilarity
Responsibilities:	**Collaborations:**	
Define similarity interface	JaccardSimilarity	
Enable interchangeable strategies	OverlapSimilarity	

Class name: JaccardSimilarity	**Superclass:** SimilarityMeasure	**Subclasses: —**
Responsibilities:	**Collaborations:**	
Compute Jaccard similarity	SimilarityMeasure	
Use intersection and union		

Class name: OverlapSimilarity	**Superclass:** SimilarityMeasure	**Subclasses: —**
Responsibilities:	**Collaborations:**	
Compute overlap similarity	SimilarityMeasure	
Use intersection and minimum set		

Class name: Flask Application	**Superclass: —**	**Subclasses: —**
Responsibilities:	**Collaborations:**	
Handle HTTP requests	Ontology	
Route user queries	AnnotationSet	
Render templates	JaccardSimilarity	
Provide gene search		
Provide GO search	OverlapSimilarity	
Provide similarity results		